

Context-Free Grammars

Generating Recursive Structure

Sheikh Azizul Hakim

CSE 211: Theory of Computation

Department of Computer Science and Engineering, BUET

Where We Are Going



Finite automata recognize patterns with finite memory.

Context-free grammars generate strings with recursive, nested, or matched structure.

Guiding question

How can a finite set of rules generate infinitely many structured strings?

Learning Goals

By the end of this lecture, you should be able to:

1. define a context-free grammar formally;
2. perform derivations using grammar rules;
3. distinguish derivations, leftmost derivations, rightmost derivations, and parse trees;
4. design CFGs for diverse recursive languages;
5. identify and repair common ambiguity in grammars;
6. simplify grammars by removing useless, nullable, and unit productions;
7. convert a grammar to Chomsky Normal Form;
8. run the CYK membership algorithm on a small example.

Why Grammars?

From Recognition to Generation

A finite automaton asks:

Is this string accepted?

A grammar asks:

How could this string be built?

Recognizer view

Read input from left to right and decide yes/no.

Generator view

Start from a symbol and repeatedly expand it until only terminals remain.

The generator view is natural for syntax: programs, expressions, nested calls, trees, brackets, and protocols.

Recursive Objects Are Everywhere

Nested brackets

`([]{}))`

A valid object may contain smaller valid objects.

Function calls

`encrypt(hash(msg))`

An argument may itself be a call.

Mirrored packets

`101#101`

The second part checks the first part in reverse.

All three examples require a rule like:

a large valid object contains smaller valid objects.

A Grammar Before the Definition

Consider properly nested parentheses:

$\varepsilon, \quad (), \quad ()(), \quad (()), \quad (()()), \quad (((()))())$

A compact set of construction rules is:

$$S \rightarrow SS \mid (S) \mid \varepsilon.$$

$$S \rightarrow \varepsilon$$

The empty string is balanced.

$$S \rightarrow (S)$$

Wrapping a balanced string keeps it balanced.

$$S \rightarrow SS$$

Concatenating two balanced strings keeps it balanced.

Worked Derivation: Generating $((()))$

Using

$$S \rightarrow SS \mid (S) \mid \varepsilon,$$

we can derive:

$$\begin{aligned} S &\Rightarrow (S) \\ &\Rightarrow (SS) \\ &\Rightarrow ((S)S) \\ &\Rightarrow (()S) \\ &\Rightarrow (() (S)) \\ &\Rightarrow (()()). \end{aligned}$$

Observation

A derivation is a construction history. The same final string may sometimes have more than one construction history.

What a Grammar Is Good At

A grammar is good when strings can be built by repeatedly applying a few local construction principles.



Key intuition

A CFG has only finitely many rules, but recursion lets those rules be reused arbitrarily many times.

Formal Definition

Formal Definition of a CFG

A context-free grammar is a 4-tuple

$$G = (V, \Sigma, R, S),$$

where:

- V is a finite set of **variables** or **nonterminals**;
- Σ is a finite set of **terminals**, disjoint from V ;
- R is a finite set of productions;
- $S \in V$ is the start variable.

Each production has the form

$$A \rightarrow \alpha,$$

where $A \in V$ and $\alpha \in (V \cup \Sigma)^*$.

Variables Versus Terminals

Variables

Variables are placeholders for structured pieces not yet fully expanded.

$S, E, T, Factor, Stmt$

Terminals

Terminals are the actual symbols that appear in the final string.

$0, 1, +, *, (,), \text{if}, \text{else}$

A derivation ends successfully only when no variables remain.

$$S \xRightarrow{*} w, \quad w \in \Sigma^*.$$

Example Grammar as a 4-Tuple

For mixed brackets, let

$$S \rightarrow SS \mid (S) \mid [S] \mid \{S\} \mid \varepsilon.$$

Then

$$G_{br} = (V, \Sigma, R, S),$$

where

$$V = \{S\},$$

$$\Sigma = \{ (,), [,], \{, \} \},$$

and R is the set of five productions above.

Language generated

$$L(G_{br}) = \{w \mid w \text{ is a properly nested mixed-bracket string}\}.$$

Reading Production Rules

A rule

$$A \rightarrow \alpha$$

means:

Whenever the current sentential form contains A , we may replace that occurrence by α .

For example, if

$$S \rightarrow 0S1 \mid \#,$$

then

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 00\#11.$$

Convention

Unless otherwise stated, the variable on the left-hand side of the first listed production is considered *the start variable*.

Context-free means

The rule for replacing A does not depend on the symbols surrounding that occurrence of A .

Sentential Forms and Derivations

A **sentential form** is any string over variables and terminals that can appear during a derivation.

For the grammar

$$S \rightarrow 0S1 \mid \#,$$

we have:

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 00\#11.$$

Step	Sentential form
0	S
1	$0S1$
2	$00S11$
3	$00\#11$

The final line is a terminal string.

The Language of a Grammar

For a CFG $G = (V, \Sigma, R, S)$, the language of G is

$$L(G) = \{w \in \Sigma^* \mid S \xRightarrow{*} w\}.$$

A language L is **context-free** if there exists some CFG G such that

$$L = L(G).$$

Important distinction

A grammar is a description. A language is the set of strings described by the grammar.

Derivations

A Tiny Grammar for Nested Commands

Let us generate command strings such as

`encrypt(sign(data)).`

Use the grammar:

$$S \rightarrow D \mid F(S),$$
$$F \rightarrow \text{hash} \mid \text{sign} \mid \text{encrypt},$$
$$D \rightarrow \text{data} \mid \text{msg} \mid \text{key}.$$

Interpretation

A command is either raw data, or a function applied to a smaller command.

Worked Derivation: `encrypt(sign(data))`

$S \Rightarrow F(S)$

$\Rightarrow \text{encrypt}(S)$

$\Rightarrow \text{encrypt}(F(S))$

$\Rightarrow \text{encrypt}(\text{sign}(S))$

$\Rightarrow \text{encrypt}(\text{sign}(D))$

$\Rightarrow \text{encrypt}(\text{sign}(\text{data})).$

What recursion did

The outer command was generated first, but it forced the inside to be another valid command.

Leftmost and Rightmost Derivations

A sentential form may contain several variables.

Leftmost derivation

At each step, replace the leftmost variable.

$\Rightarrow_{lm}$

Rightmost derivation

At each step, replace the rightmost variable.

$\Rightarrow_{rm}$

Important

Leftmost and rightmost derivations control the order of expansion. They do not necessarily change the final parse tree.

Worked Leftmost Derivation

Use

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid a.$$

A leftmost derivation of $a + a * a$:

$$\begin{aligned} E &\Rightarrow_{\text{lm}} E + T \\ &\Rightarrow_{\text{lm}} T + T \\ &\Rightarrow_{\text{lm}} F + T \\ &\Rightarrow_{\text{lm}} a + T \\ &\Rightarrow_{\text{lm}} a + T * F \\ &\Rightarrow_{\text{lm}} a + F * F \\ &\Rightarrow_{\text{lm}} a + a * F \\ &\Rightarrow_{\text{lm}} a + a * a. \end{aligned}$$

Worked Rightmost Derivation

Using the same grammar:

$$\begin{aligned} E &\Rightarrow_{\text{rm}} E + T \\ &\Rightarrow_{\text{rm}} E + T * F \\ &\Rightarrow_{\text{rm}} E + T * a \\ &\Rightarrow_{\text{rm}} E + F * a \\ &\Rightarrow_{\text{rm}} E + a * a \\ &\Rightarrow_{\text{rm}} T + a * a \\ &\Rightarrow_{\text{rm}} F + a * a \\ &\Rightarrow_{\text{rm}} a + a * a. \end{aligned}$$

Same string, same intended parse

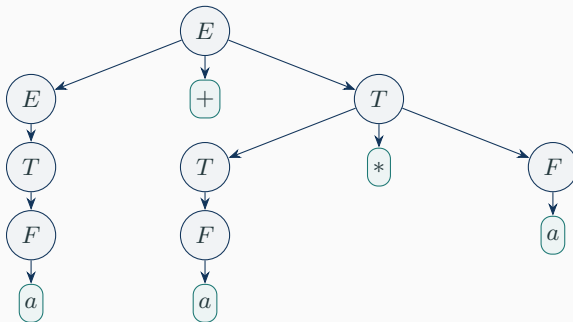
The expansion order changed, but the multiplication still binds inside the final T .

Derivations Are Linear; Structure Is a Tree

A derivation is written as a sequence.

$$E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T \Rightarrow \dots$$

But the object being built is hierarchical.



Parse Trees

Parse Trees: Formal Conditions

For a CFG $G = (V, \Sigma, R, S)$, a parse tree satisfies:

1. The root is labeled by the start variable S .
2. Every internal node is labeled by a variable.
3. Every leaf is labeled by a terminal or by ε .
4. If an internal node labeled A has children, from left to right, labeled

$$X_1, X_2, \dots, X_k,$$

then

$$A \rightarrow X_1 X_2 \cdots X_k$$

is a production of the grammar.

Special case

If a node labeled A uses the production $A \rightarrow \varepsilon$, then it has a single child labeled ε .

Yield of a Parse Tree

The **yield** of a parse tree is obtained by reading its leaves from left to right, ignoring any ε -leaf.

Grammar

$$S \rightarrow SS \mid (S) \mid \varepsilon$$

For the string $(())$, use:

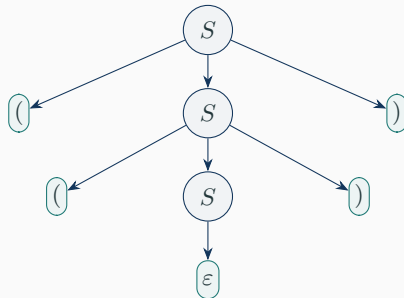
$$S \Rightarrow (S) \Rightarrow ((S)) \Rightarrow (()).$$

The leaves, read from left to right, are:

$(, (, \varepsilon,),)$.

Therefore,

$$\text{yield} = (()).$$



Parse Tree for a Nested Command

Grammar

$$S \rightarrow D \mid F(S),$$
$$F \rightarrow \text{hash} \mid \text{sign} \mid \text{encrypt},$$
$$D \rightarrow \text{data} \mid \text{msg} \mid \text{key}.$$

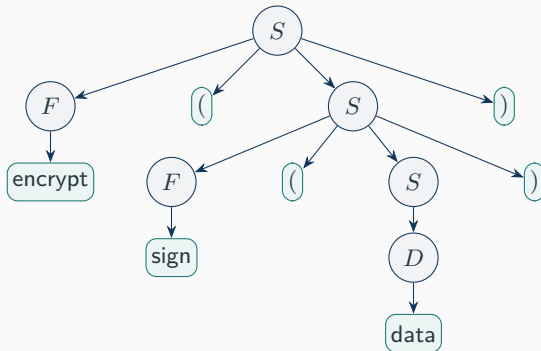
For the string

`encrypt(sign(data)),`

a derivation is:

$$S \Rightarrow F(S) \Rightarrow \text{encrypt}(S)$$
$$\Rightarrow \text{encrypt}(F(S)) \Rightarrow \text{encrypt}(\text{sign}(S))$$
$$\Rightarrow \text{encrypt}(D) \Rightarrow \text{encrypt}(\text{sign}(\text{data})).$$

yield = `encrypt(sign(data))`



Derivations and Parse Trees Are Equivalent

For every variable A and terminal string w , the following are equivalent:

1. $A \xRightarrow{*} w$.
2. $A \xRightarrow{*}_{\text{lm}} w$.
3. $A \xRightarrow{*}_{\text{rm}} w$.
4. There is a parse tree with root A and yield w .



The proofs are by induction on derivation length or tree height.

Recursive Inference: Bottom-Up Thinking

Parse trees can be read top-down or bottom-up.

Top-down

Start from S , expand variables, and try to generate the input.

Bottom-up

Start from small substrings and infer which variables could have generated them.

The CYK algorithm at the end of the lecture is a systematic bottom-up parser for grammars in CNF.

Designing CFGs: Worked Examples

When designing a grammar, ask:

1. What is the smallest valid string?
2. How can a larger valid string be built from smaller valid strings?
3. Is there a natural first/last pair to generate together?
4. Is the language a union of simpler languages?
5. Does part of the language happen to be regular?
6. Which variables correspond to meaningful syntactic categories?
7. Why can no invalid string be generated?

Example 1: Balanced Mixed Brackets

$L_1 = \{w \mid w \text{ is a properly nested string over } (), [], \{\}\}.$

Grammar

$$S \rightarrow SS \mid (S) \mid [S] \mid \{S\} \mid \varepsilon.$$

Construction

Wrap a balanced object, or concatenate two balanced objects.

Sample strings

$\varepsilon, \quad (), \quad ([]), \quad []\{()\}$

Worked Derivation: $[]\{()\}$

$$S \rightarrow SS \mid (S) \mid [S] \mid \{S\} \mid \varepsilon.$$

$$\begin{aligned} S &\Rightarrow SS \\ &\Rightarrow [S]S \\ &\Rightarrow []S \\ &\Rightarrow []\{S\} \\ &\Rightarrow []\{(S)\} \\ &\Rightarrow []\{()\}. \end{aligned}$$

Rejection intuition

Strings like $([])$ cannot be generated because every opening bracket must be closed by the matching wrapper rule.

Why Example 1 Works

Soundness

Every rule preserves proper nesting:

- ε is balanced;
- (S) , $[S]$, $\{S\}$ wrap a balanced string with matching brackets;
- SS concatenates two balanced strings.

Completeness

Every nonempty balanced string is either:

- one bracket pair surrounding a balanced inside; or
- a concatenation of two shorter balanced strings.

Example 2: Equal Numbers of 0s and 1s in Any Order

$$L_2 = \{w \in \{0, 1\}^* \mid \#_0(w) = \#_1(w)\}.$$

A grammar is:

$$S \rightarrow 0S1S \mid 1S0S \mid \varepsilon.$$

Why this is interesting

The matching symbols need not be adjacent or appear in two blocks.

$$001011, \quad 0101, \quad 1010 \in L_2.$$

Not merely $0^n 1^n$

This language allows arbitrary interleavings as long as the two counts agree.

Worked Derivation: 0101

Using

$$S \rightarrow 0S1S \mid 1S0S \mid \varepsilon,$$

we derive:

$$\begin{aligned} S &\Rightarrow 0S1S \\ &\Rightarrow 01S \\ &\Rightarrow 010S1S \\ &\Rightarrow 0101S \\ &\Rightarrow 0101. \end{aligned}$$

Invariant

Each nonempty production introduces one 0 and one 1. Therefore every generated terminal string has equal counts.

Why the Grammar for Equal Counts Is Complete

Take a nonempty balanced string w . Suppose it starts with 0.

Scan from left to right until the first point where the number of 0's and 1's in the prefix becomes equal.

Then

$$w = 0x1y$$

where x and y are both balanced.

So the rule

$$S \rightarrow 0S1S$$

can generate it recursively.

Symmetric case

If w starts with 1, use $S \rightarrow 1S0S$.

Example 3: Mirrored Packets

$$L_3 = \{w\#w^R \mid w \in \{0,1\}^*\}.$$

Grammar

$$S \rightarrow 0S0 \mid 1S1 \mid \#.$$

Construction

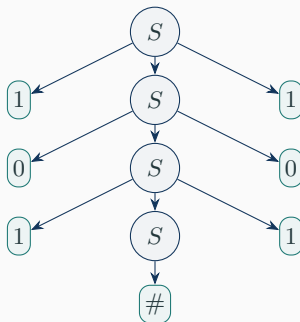
Choose the middle marker $\#$, then wrap equal symbols around it.

Examples

$\#$, $0\#0$, $10\#01$, $101\#101$.

Worked Derivation: 101#101

$S \Rightarrow 1S1$
 $\Rightarrow 10S01$
 $\Rightarrow 101S101$
 $\Rightarrow 101\#101.$



Example 4: All Palindromes

$$L_4 = \{w \in \{0, 1\}^* \mid w = w^R\}.$$

Grammar

$$P \rightarrow 0P0 \mid 1P1 \mid 0 \mid 1 \mid \varepsilon.$$

Even-length base

$$P \rightarrow \varepsilon$$

Examples: $\varepsilon, 00, 0110$.

Odd-length base

$$P \rightarrow 0 \mid 1$$

Examples: $0, 1, 10101$.

Example 5: Two Independent Matching Layers

$$L_5 = \{a^n b^m c^m d^n \mid n, m \geq 0\}.$$

Grammar

$$S \rightarrow aSd \mid T, \quad T \rightarrow bTc \mid \varepsilon.$$

Idea

The outer recursion matches a 's with d 's. The inner recursion matches b 's with c 's.

$$\underbrace{a^n}_{S \text{ wraps}} \underbrace{b^m c^m}_{T \text{ generates}} \underbrace{d^n}_{S \text{ wraps}}$$

Worked Derivation: $a^2b^3c^3d^2$

$$S \Rightarrow aSd$$

$$\Rightarrow aaSdd$$

$$\Rightarrow aaTdd$$

$$\Rightarrow aabTcdd$$

$$\Rightarrow aabbTccdd$$

$$\Rightarrow aabbbTcccdd$$

$$\Rightarrow aabbbcccd.$$

Design pattern

Use one variable for each independent obligation, and nest variables when one equality must sit inside another.

Example 6: One of Two Equalities

$$L_6 = \{a^i b^j c^k \mid i = j \text{ or } j = k\}.$$

Build the language as a union:

$$S \rightarrow S_1 \mid S_2.$$

Branch 1: verify $i = j$

$$S_1 \rightarrow AC, \quad A \rightarrow aAb \mid \varepsilon, \quad C \rightarrow cC \mid \varepsilon.$$

Branch 2: verify $j = k$

$$S_2 \rightarrow DB, \quad D \rightarrow aD \mid \varepsilon, \quad B \rightarrow bBc \mid \varepsilon.$$

Example 7: Unequal Numbers of 0s and 1s

$$L_7 = \{w \in \{0, 1\}^* \mid \#_0(w) \neq \#_1(w)\}.$$

Grammar

$$\begin{aligned}U &\rightarrow A \mid B, \\A &\rightarrow E0E \mid E0A, \\B &\rightarrow E1E \mid E1B, \\E &\rightarrow 0E1E \mid 1E0E \mid \varepsilon.\end{aligned}$$

Meaning of the variables

- E generates strings with equal numbers of 0's and 1's.
- A generates strings with more 0's than 1's.
- B generates strings with more 1's than 0's.

Worked derivation

For 0010, which has three 0's and one 1:

$$\begin{aligned}U &\Rightarrow A \Rightarrow E0A \Rightarrow 0A \\&\Rightarrow 0E0E \Rightarrow 00E \\&\Rightarrow 001E0E \Rightarrow 0010.\end{aligned}$$

Example 8: Binary Trees in Prefix Form

Let x be a leaf and $n(T, T)$ be an internal node with two children.

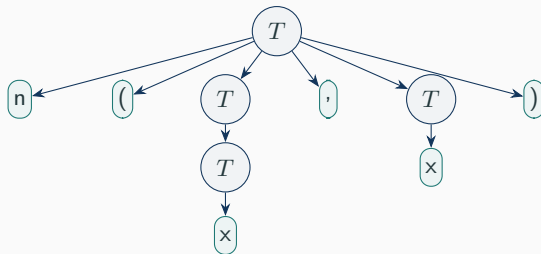
$$T \rightarrow x \mid n(T, T).$$

Generated strings

x

$n(x, x)$

$n(n(x, x), x)$



Example 9: A Tiny Query Language

Generate queries such as

```
select(name,where(age>18)).
```

$$Q \rightarrow \text{select}(F, W),$$
$$F \rightarrow \text{name} \mid \text{age} \mid \text{dept},$$
$$W \rightarrow \text{all} \mid \text{where}(P),$$
$$P \rightarrow F \ O \ V,$$
$$O \rightarrow > \mid = \mid < ,$$
$$V \rightarrow 18 \mid 30 \mid \text{cse}.$$

Moral

Variables often represent syntactic categories, not just counting devices.

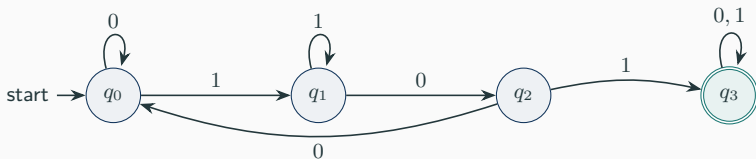
Worked Derivation: Query Language

$Q \Rightarrow \text{select}(F, W)$
 $\Rightarrow \text{select}(\text{name}, W)$
 $\Rightarrow \text{select}(\text{name}, \text{where}(P))$
 $\Rightarrow \text{select}(\text{name}, \text{where}(FOV))$
 $\Rightarrow \text{select}(\text{name}, \text{where}(\text{age}OV))$
 $\Rightarrow \text{select}(\text{name}, \text{where}(\text{age} > V))$
 $\Rightarrow \text{select}(\text{name}, \text{where}(\text{age} > 18)).$

Example 10: Converting a DFA to a CFG

Let

$$L_{10} = \{w \in \{0, 1\}^* \mid w \text{ contains the substring } 101\}.$$



Make one variable R_i for each state q_i .

DFA-to-CFG Construction

For every transition $\delta(q_i, a) = q_j$, add

$$R_i \rightarrow aR_j.$$

For every accept state q_i , add

$$R_i \rightarrow \varepsilon.$$

The start variable is the variable corresponding to the DFA start state.

For the substring 101 DFA:

$$R_0 \rightarrow 0R_0 \mid 1R_1,$$

$$R_1 \rightarrow 1R_1 \mid 0R_2,$$

$$R_2 \rightarrow 0R_0 \mid 1R_3,$$

$$R_3 \rightarrow 0R_3 \mid 1R_3 \mid \varepsilon.$$

Every Regular Language Is Context-Free

Theorem

Every regular language is a context-free language.

Proof

Let L be regular. Then some DFA

$$M = (Q, \Sigma, \delta, q_0, F)$$

recognizes L .

Construct a CFG G as follows:

- create one variable R_q for every state $q \in Q$;
- use R_{q_0} as the start variable;
- for every transition $\delta(q, a) = r$, add $R_q \rightarrow aR_r$;
- for every accepting state $q \in F$, add $R_q \rightarrow \varepsilon$.

A derivation $R_{q_0} \xRightarrow{*} wR_q$ exists exactly when the DFA reaches q after reading w . Therefore, $R_{q_0} \xRightarrow{*} w$ exactly when the DFA reaches an accepting state after reading w . Hence, $L(G) = L(M) = L$.

Thus, every regular language is context-free.

Example 11: A Tempting Non-Example

Consider

$$L = \{ww \mid w \in \{0,1\}^*\}.$$

Students often try:

$$S \rightarrow 0S0 \mid 1S1 \mid \varepsilon.$$

But this generates

$$\{ww^R \mid w \in \{0,1\}^*\},$$

not $\{ww\}$.

Important intuition

Recursive wrapping remembers the first half in reverse order. This is exactly why stacks naturally recognize mirrors, not copies.

Design Patterns We Have Used

Pattern	Typical rule form
Wrapping	$S \rightarrow aSb$
Concatenation	$S \rightarrow SS$ or $S \rightarrow AB$
Optional structure	$A \rightarrow \alpha \mid \varepsilon$
Union of cases	$S \rightarrow S_1 \mid S_2$
Regular part	Convert DFA states into variables
Syntactic categories	One variable per category

Ambiguity

Ambiguous Grammars

A string w is **derived ambiguously** in a CFG G if it has two or more different parse trees.

Equivalently, it has:

- two or more different leftmost derivations; or
- two or more different rightmost derivations.

A grammar G is **ambiguous** if it generates at least one string ambiguously.

Ambiguity is about grammar, not just language

The same language may have both ambiguous and unambiguous grammars.

Ambiguous Expression Grammar

Consider

$$E \rightarrow E + E \mid E * E \mid (E) \mid a.$$

The string

$a+a*a$

can be parsed as either

$a + a * a$

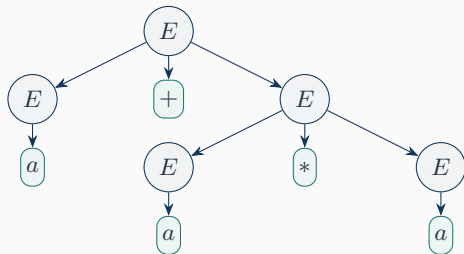
or

$a + a * a.$

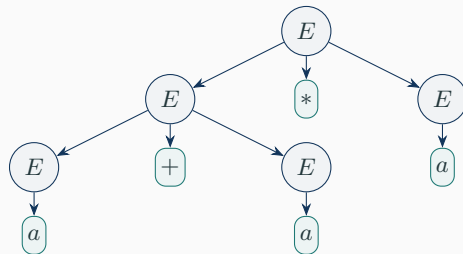
Problem

The grammar says that $+$ and $*$ are structurally equal, so it fails to encode precedence.

Two Parse Trees for $a + a * a$



$a + a * a$



$a + a * a$

Removing Expression Ambiguity

We introduce levels:

- F : factor, cannot be split by an adjacent operator;
- T : term, product of factors;
- E : expression, sum of terms.

$$E \rightarrow E + T \mid T,$$

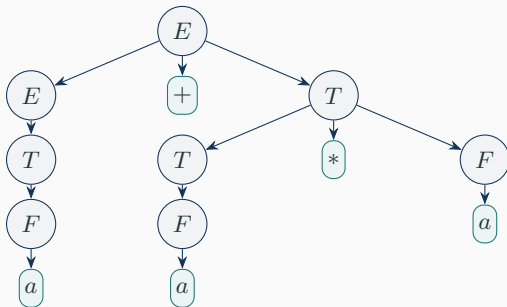
$$T \rightarrow T * F \mid F,$$

$$F \rightarrow (E) \mid a.$$

Result

Multiplication is forced below addition in the tree, so $*$ binds more tightly than $+$.

Parse Tree After Removing Ambiguity



$a+a*a$ is forced to mean $a+a*a$.

Ambiguity Example: Dangling Else

A naive statement grammar:

$$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{other.}$$

The string

`if E then if E then other else other`

can attach else to:

Inner if

`if E then (if E then other else other)`

Outer if

`if E then (if E then other) else other`

Repairing Dangling Else

Separate matched and unmatched statements.

$$S \rightarrow M \mid U,$$
$$M \rightarrow \text{other} \mid \text{if } E \text{ then } M \text{ else } M,$$
$$U \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } U.$$

Convention encoded

Every else attaches to the nearest unmatched if.

Structural Ambiguity in Balanced Parentheses

The grammar

$$S \rightarrow SS \mid (S) \mid \varepsilon$$

correctly generates balanced parentheses, but it is ambiguous.

For example:

$() () ()$

can be grouped as

$() () ()$

or as

$() () ().$

Lesson

A language may have an obvious structure, but a grammar may still give multiple parse trees for concatenation.

Inherent Ambiguity

A CFL L is **inherently ambiguous** if every CFG for L is ambiguous.

A standard example is:

$$L = \{a^n b^n c^m d^m \mid n, m \geq 1\} \\ \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}.$$

Why ambiguity is unavoidable

Strings of the form

$$a^n b^n c^n d^n$$

belong to both sides of the union.

Grammar for the Inherently Ambiguous Example

$$\begin{aligned} S &\rightarrow AB \mid C, \\ A &\rightarrow aAb \mid ab, \\ B &\rightarrow cBd \mid cd, \\ C &\rightarrow aCd \mid aDd, \\ D &\rightarrow bDc \mid bc. \end{aligned}$$

Branch AB

Matches a 's with b 's and c 's with d 's.

Branch C

Matches outer a, d and inner b, c .

Pumping Lemma for Context-Free Languages

Limits of Context-Free Grammars

Context-free grammars can generate many recursive structures:

$$a^n b^n, \quad w c w^R, \quad a^n b^m c^m d^n.$$

However, some languages require more coordination than one context-free derivation can maintain.

Examples include:

$$\{a^n b^n c^n \mid n \geq 0\},$$
$$\{a^n b^m c^n d^m \mid n, m \geq 0\},$$

and

$$\{a^q \mid q \text{ is prime}\}.$$

Main question

How can we prove that no context-free grammar generates a given language?

The Pumping Idea

A sufficiently long string generated by a CFG must have a sufficiently tall parse tree.

Since the grammar contains only finitely many variables, some variable must repeat along a root-to-leaf path.

$$A \Rightarrow^* vAy \quad \text{and} \quad A \Rightarrow^* x.$$

The repeated part of the tree can be used again:

$$A \Rightarrow^* vAy \Rightarrow^* vvAyy \Rightarrow^* vvvAyyy \Rightarrow^* \dots$$

Thus, two parts of the generated string can be repeated together:

$$v \quad \text{and} \quad y.$$

Pumping Lemma for Context-Free Languages

If L is context-free, then there exists an integer $p \geq 1$ such that every string $w \in L$ with $|w| \geq p$ can be written as

$$w = uvxyz$$

satisfying:

1. $|vxy| \leq p$;
2. $|vy| > 0$;
3. for every $i \geq 0$, $uv^i xy^i z \in L$.

The integer p is called the **pumping length**.

Understanding the Five Parts

The decomposition is

$$w = \underbrace{u}_{\text{unchanged}} \underbrace{v}_{\text{pumped}} \underbrace{x}_{\text{unchanged}} \underbrace{y}_{\text{pumped}} \underbrace{z}_{\text{unchanged}} .$$

Different pumping values produce:

i	$uv^i xy^i z$
0	uxz
1	$uvxyz = w$
2	$uvvxyyz$
3	$uvvvxyyyz$

Locality

$$|vxy| \leq p$$

places both pumped pieces inside one short region.

Nonempty change

$$|vy| > 0$$

ensures that pumping changes the string.

The Quantifier Order

The Pumping Lemma states:

$$\exists p \forall w \exists u, v, x, y, z \forall i \geq 0.$$

More explicitly:

- the language determines a pumping length p ;
- we may choose a sufficiently long string w ;
- the decomposition may be chosen adversarially;
- every pumping value must preserve membership.

To prove that L is not context-free, we show:

$$\forall p \exists w \in L \forall \text{ valid decompositions } \exists i \geq 0$$

such that

$$uv^i xy^i z \notin L.$$

Who Chooses What?

Our choices

We choose:

- a witness string w ;
- usually based on p ;
- a pumping value i , often 0 or 2.

Adversary's choice

The adversary chooses:

$$w = uvxyz$$

subject only to

$$|vxy| \leq p \quad \text{and} \quad |vy| > 0.$$

Therefore, a proof must handle **every valid decomposition**, not only one convenient decomposition.

Standard Proof Template

To prove that L is not context-free:

1. Assume, for contradiction, that L is context-free.
2. Let p be the pumping length.
3. Choose

$$w \in L \quad \text{with} \quad |w| \geq p.$$

4. Consider an arbitrary valid decomposition

$$w = uvxyz.$$

Use $|vxy| \leq p$ to restrict the possible positions of v and y .

5. Choose a suitable $i \geq 0$. Prove that

$$uv^i xy^i z \notin L.$$

Conclude that L is not context-free.

Choosing a Good Witness String

The witness should expose the property that pumping cannot preserve.

Required property	Typical witness
Three equal blocks	$a^p b^p c^p$
Two crossing equalities	$a^p b^p c^p d^p$
Prime length	a^q , where $q > p$ is prime
Nonlinear relationship	$a^p b^{p^2}$

Main design principle

Choose block lengths using p , so that the restriction

$$|vxy| \leq p$$

forces the pumping to affect only a limited part of the string.

Example 1: Three Equal Blocks

Consider

$$L_1 = \{a^n b^n c^n \mid n \geq 0\}.$$

Assume, for contradiction, that L_1 is context-free.

Let p be its pumping length and choose

$$w = a^p b^p c^p.$$

Clearly,

$$w \in L_1 \quad \text{and} \quad |w| = 3p \geq p.$$

Consider an arbitrary decomposition

$$w = uvxyz$$

satisfying

$$|vxy| \leq p, \quad |vy| > 0.$$

Example 1: Where Can vxy Occur?

The witness consists of three blocks:

$$\underbrace{a^p}_p \underbrace{b^p}_p \underbrace{c^p}_p.$$

Because

$$|vxy| \leq p,$$

the substring vxy cannot contain both an a and a c .

To do so, it would have to include:

- at least one a ;
- all p symbols in the b -block;
- at least one c .

Therefore,

$$|vxy| \geq p + 2,$$

which is impossible.

Hence, vxy touches at most two adjacent blocks.

Example 1: Pumping Across a Boundary

Pump with

$$i = 2.$$

Suppose v or y contains two different symbol types.

For example, suppose

$$v = a^r b^s$$

where $r, s > 0$.

Then pumping produces

$$v^2 = a^r b^s a^r b^s,$$

which contains an a after a b .

Thus, the pumped string is no longer of the form

$$a^* b^* c^*.$$

The case where a pumped piece contains both b 's and c 's is symmetric.

Example 1: Pumping Inside the Blocks

Now suppose each of v and y lies entirely inside one symbol block.

Since vy touches at most two adjacent blocks, pumping changes the lengths of at most two blocks.

The third block remains of length p .

Because

$$|vy| > 0,$$

at least one of the affected block lengths increases when $i = 2$.

Therefore, the three block lengths cannot all remain equal:

$$\#_a \neq \#_b \quad \text{or} \quad \#_b \neq \#_c.$$

Hence,

$$uv^2xy^2z \notin L_1.$$

Example 1: Conclusion

For every valid decomposition, pumping with $i = 2$ produces a string that either:

- is not of the form $a^*b^*c^*$; or
- does not contain equal numbers of a 's, b 's, and c 's.

Thus,

$$uv^2xy^2z \notin L_1.$$

This contradicts the Pumping Lemma.

Therefore,

$$\{a^n b^n c^n \mid n \geq 0\} \text{ is not context-free.}$$

Example 2: Crossing Dependencies

Consider

$$L_2 = \{a^n b^m c^n d^m \mid n, m \geq 0\}.$$

The language requires:

$$\#_a = \#_c \quad \text{and} \quad \#_b = \#_d.$$

These dependencies cross:

$$\underbrace{a^n}_{\text{matches } c^n} \underbrace{b^m}_{\text{matches } d^m} \underbrace{c^n}_{\text{matches } a^n} \underbrace{d^m}_{\text{matches } b^m}.$$

Assume that L_2 is context-free and let p be its pumping length.

Choose

$$w = a^p b^p c^p d^p.$$

Example 2: Locality Versus Crossing Matches

Consider an arbitrary valid decomposition

$$w = uvxyz.$$

Since

$$|vxy| \leq p,$$

the pumped region touches at most two adjacent blocks.

However, the blocks that must remain equal are nonadjacent:

$$a \leftrightarrow c, \quad b \leftrightarrow d.$$

Pump with

$$i = 2.$$

If a pumped piece crosses a block boundary, repetition destroys the order

$$a^* b^* c^* d^*.$$

Otherwise, pumping changes one or two adjacent blocks while leaving their nonadjacent matching blocks unchanged.

Example 2: Contradiction

Because

$$|vy| > 0,$$

at least one block length changes.

But the corresponding nonadjacent matching block is outside vxy and remains unchanged.

Therefore, after pumping,

$$\#_a \neq \#_c \quad \text{or} \quad \#_b \neq \#_d.$$

Hence,

$$uv^2xy^2z \notin L_2.$$

This contradicts the Pumping Lemma.

Therefore,

$$\{a^n b^m c^n d^m \mid n, m \geq 0\} \text{ is not context-free.}$$

Example 3: Prime-Length Strings

Consider the unary language

$$L_3 = \{a^q \mid q \text{ is prime}\}.$$

Thus,

$$a^2, a^3, a^5, a^7, a^{11}, \dots \in L_3.$$

Assume, for contradiction, that L_3 is context-free.

Let p be its pumping length.

Choose a prime number q such that

$$q > p,$$

and let

$$w = a^q.$$

Then

$$w \in L_3 \quad \text{and} \quad |w| = q \geq p.$$

Example 3: Measuring the Pumped Parts

Consider an arbitrary valid decomposition

$$w = uvxyz.$$

Let

$$d = |v| + |y|.$$

From

$$|vy| > 0,$$

we obtain

$$d \geq 1.$$

Also,

$$d \leq |vxy| \leq p < q.$$

Therefore,

$$1 \leq d < q.$$

Since the alphabet contains only a , pumping changes only the total length of the string.

Example 3: Pumping to a Composite Length

Choose

$$i = q + 1.$$

The original string has length q . Pumping adds q additional copies of both v and y .

Therefore,

$$\begin{aligned} |uv^{q+1}xy^{q+1}z| &= q + q(|v| + |y|) \\ &= q + qd \\ &= q(1 + d). \end{aligned}$$

Since

$$q \geq 2 \quad \text{and} \quad d \geq 1,$$

both factors in

$$q(1 + d)$$

are greater than 1.

Thus, the pumped length is composite.

Example 3: Conclusion

The pumped string is

$$uv^{q+1}xy^{q+1}z = a^{q(1+d)}.$$

Its length is composite, so

$$uv^{q+1}xy^{q+1}z \notin L_3.$$

This contradicts the Pumping Lemma.

Therefore,

$$\{a^q \mid q \text{ is prime}\} \text{ is not context-free.}$$

Additional consequence

Since every regular language is context-free, this also proves that the prime-length language is not regular.

Example 4: Quadratic Growth

Consider

$$L_4 = \{a^n b^{n^2} \mid n \geq 0\}.$$

Assume that L_4 is context-free and let p be its pumping length.

Choose

$$w = a^p b^{p^2}.$$

Consider an arbitrary decomposition

$$w = uvxyz$$

satisfying

$$|vxy| \leq p, \quad |vy| > 0.$$

Define

$$\alpha = \#_a(vy), \quad \beta = \#_b(vy).$$

Example 4: Effects of Pumping

Since

$$|vy| > 0,$$

we have

$$\alpha + \beta > 0.$$

Also,

$$\alpha + \beta = |vy| \leq p.$$

Pump with

$$i = 2.$$

If v or y contains both a 's and b 's, repeating that piece creates a b before a later a .

The resulting string is not of the form

$$a^*b^*.$$

Otherwise, the pumped string has the form

$$a^{p+\alpha}b^{p^2+\beta}.$$

Example 4: Required Equality

For

$$a^{p+\alpha} b^{p^2+\beta}$$

to belong to L_4 , we would need

$$p^2 + \beta = (p + \alpha)^2.$$

Expanding the right-hand side:

$$p^2 + \beta = p^2 + 2p\alpha + \alpha^2.$$

Therefore,

$$\beta = 2p\alpha + \alpha^2.$$

We now consider whether $\alpha = 0$ or $\alpha > 0$.

Example 4: Contradiction

Case 1: $\alpha = 0$

Since

$$\alpha + \beta > 0,$$

we have $\beta > 0$.

The number of a 's remains p , but the number of b 's becomes

$$p^2 + \beta \neq p^2.$$

Therefore, the pumped string is not in L_4 .

Case 2: $\alpha > 0$

Membership would require

$$\beta = 2p\alpha + \alpha^2.$$

But

$$\beta \leq p,$$

whereas

$$2p\alpha + \alpha^2 > p$$

Example 4: Conclusion

Every valid decomposition fails when pumped with $i = 2$.

Therefore,

$$uv^2xy^2z \notin L_4,$$

contradicting the Pumping Lemma.

Hence,

$$\{a^n b^{n^2} \mid n \geq 0\} \text{ is not context-free.}$$

Main obstruction

Pumping changes symbol counts by fixed linear amounts, but the language requires the nonlinear relationship

$$\#_b = (\#_a)^2.$$

Choosing the Pumping Value

The most common choices are $i = 0$ and $i = 2$.

Pump down: $i = 0$

$$uv^0xy^0z = uxz.$$

Useful when removing symbols breaks:

- equality between blocks;
- a required component;
- a minimum-length condition.

Pump up: $i = 2$

$$uv^2xy^2z.$$

Useful when repetition creates:

- incorrect symbol order;
- unequal block lengths;
- excessive growth.

A larger value may also be useful, as in the prime-length example.

Common Mistake: Choosing the Decomposition

An incorrect proof may say:

Choose $v = a$ and $y = b$. Pumping destroys the equality.

The proof is invalid because we do not control the decomposition.

The adversary may place v and y anywhere, provided that

$$|vxy| \leq p \quad \text{and} \quad |vy| > 0.$$

Correct wording

Consider an arbitrary valid decomposition

$$w = uvxyz.$$

Then show that every possible placement of v and y leads to a contradiction.

Common Mistake: Pumping Only One Piece

The CFL Pumping Lemma pumps both v and y :

$$uv^i xy^i z.$$

It is not sufficient to analyze only v .

For example:

- v may add a 's;
- y may simultaneously add b 's;
- their combined effect determines whether an equality is preserved.

Useful quantities include:

$$\#_a(vy), \quad \#_b(vy), \quad |v| + |y|.$$

Remember that v or y may individually be empty, but

$$|vy| > 0$$

ensures that they are not both empty.

Common Mistake: Incomplete Case Analysis

For

$$a^p b^p c^p,$$

the region vxy may lie:

- inside the a -block;
- inside the b -block;
- inside the c -block;
- across the a/b boundary;
- across the b/c boundary.

A complete proof must address all possibilities.

Efficient grouping

Instead of treating every position separately, group them into:

1. a pumped piece contains more than one symbol type;
2. each pumped piece remains inside a single block.

What the Lemma Does and Does Not Prove

The Pumping Lemma gives a necessary property:

$$L \text{ is context-free} \implies L \text{ satisfies the pumping property.}$$

Therefore,

$$L \text{ violates the pumping property} \implies L \text{ is not context-free.}$$

However, showing that a few selected strings can be pumped does not prove that the language is context-free.

To prove context-freeness

Construct one of the following:

- a context-free grammar;
- a pushdown automaton.

Pumping-Lemma Proof Checklist

Before completing a proof, verify:

1. Did I assume that L is context-free?
2. Did I let p be an arbitrary pumping length?
3. Is my witness w actually in L ?
4. Does $|w| \geq p$?
5. Did I consider an arbitrary valid decomposition?
6. Did I use

$$|vxy| \leq p \quad \text{and} \quad |vy| > 0?$$

7. Did I cover every possible placement of vxy ?
8. Did I explicitly choose a pumping value i ?
9. Did I prove that the pumped string is outside L ?
10. Did I state the contradiction and final conclusion?

Practice Problems

Use the Pumping Lemma to prove that the following languages are not context-free:

1.

$$\{0^n 1^n 2^n \mid n \geq 0\};$$

2.

$$\{a^n b^m c^n d^m \mid n, m \geq 0\};$$

3.

$$\{a^n b^{2n} c^{3n} \mid n \geq 0\};$$

4.

$$\{a^q \mid q \text{ is prime}\};$$

5.

$$\{a^n b^{n^3} \mid n \geq 0\}.$$

For each proof

Specify the witness, characterize every possible location of vxy , choose a pumping value, and state exactly which language condition fails.

From Limitations to Grammar Algorithms

The Pumping Lemma shows that context-free grammars have fundamental limitations.

They cannot generally maintain:

- three simultaneous equal block lengths;
- crossing equality constraints;
- prime-length requirements;
- nonlinear counting relationships.